Visión general del proyecto

Qué pedía el proyecto, qué se hizo y cómo se hizo — resumen para José y Natalia

Octubre 2026

Qué pedía el proyecto (en una página)

El PDF del proyecto ("Proyecto Apertura — DBSCAN paralelo para la detección de ruido o outliers con OpenMP") pide:

1. Implementar un algoritmo **ingenuo** (sin trucos de optimización espacial) de dos pasos para clasificar puntos como *core* o *ruido*, en **tres versiones**:
   - Una **serial** (referencia).
   - Dos **paralelas con OpenMP**: una que trata el conjunto de puntos como una sola unidad indivisible (**P1**), y otra que lo divide explícitamente en `k` bloques (**P2**).
2. Correr un **experimento de desempeño** barriendo: 8 tamaños de problema (de 20,000 a 200,000 puntos), 6 valores de `k` (solo para P2), 4 cantidades de hilos, con 10 repeticiones por configuración.
3. Entregar: todo el código, un documento explicando el código y las estrategias de paralelización, un documento con la evaluación experimental (con gráficas y una sección de uso de IA generativa), y el archivo con los datos crudos del experimento.
4. Al menos una de las dos versiones paralelas debía alcanzar un *speedup* mayor a **1.5x**, o había penalización del 50%.
5. Es trabajo en equipo de hasta 2 personas, entrega el **martes 20 de octubre**.

Cómo se hizo, paso a paso

1. Entender el algoritmo exacto que pedían

El PDF no pide el DBSCAN "completo" de libro de texto (el que asigna número de clúster a cada punto); pide una versión simplificada de dos pasos que solo distingue **core / ruido**, descrita en las láminas del PDF ("Paso 1", "Paso 2"). Esto se identificó leyendo con cuidado las láminas, porque cambia bastante lo que hay que programar.

2. Diseñar las tres versiones de código

Se escribió primero la versión **serial** (la más simple, sirve de "verdad de referencia"). Luego:

- **P1** se diseñó como la paralelización más directa posible: dejar que OpenMP reparta automáticamente los puntos entre los hilos disponibles (`#pragma omp parallel for`), sin que el programador divida nada a mano.
- **P2** se diseñó para que realmente fuera **otra estrategia**, no una copia de P1 con otro nombre: aquí el programador sí divide el conjunto de puntos en `k` bloques a mano, cada bloque se procesa por separado, y al final hay que **unir (merge)** los resultados de todos

los bloques —esto es justo lo que pregunta el PDF del proyecto: "¿cómo unir los resultados para mantener consistencia?"—.

Antes de seguir, se verificó que **las tres versiones dieran el mismo resultado** sobre un conjunto pequeño de prueba (2,000 puntos): mismo número de puntos core y de ruido. Esto es clave —no sirve de nada que una versión sea rápida si clasifica mal los puntos—.

3. Resolver el problema de los datos de entrada

El PDF dice que los puntos de prueba se deben generar con una libreta (`DBSCAN_noise.ipynb`) del repositorio del curso. Esa libreta no estaba disponible al construir el proyecto, así que se usó temporalmente un generador propio como sustituto. **El 17 de octubre se localizó la libreta real** en github.com/octavio-gutierrez/computoparalelo2026b y se comparó línea por línea contra `generar_puntos.py`: se encontraron 2 diferencias (escala de los puntos, y que la libreta no agrega ruido uniforme a mano) y se corrigió `generar_puntos.py` para usar exactamente el mismo método (`sklearn.datasets.make_blobs`). Se ajustaron `epsilon` y `min_samples` a la nueva escala y se repitió el experimento completo (2,320 corridas) con los datos ya verificados.

4. Correr el experimento completo

Aquí hubo un problema práctico: 2,320 corridas (8 tamaños × 10 repeticiones para la serial, más todas las combinaciones de P1 y P2) tardan varios minutos en total, y el entorno donde se corrió no permite dejar procesos corriendo "en segundo plano" de forma confiable entre un comando y el siguiente. La solución fue escribir un script (`run_reanudable.sh`) que:

- Revisa qué combinaciones ya tienen sus 10 repeticiones guardadas.
- Corre solo lo que falta, durante un tiempo limitado.
- Se puede volver a ejecutar las veces que haga falta, y cada vez retoma donde se quedó, hasta terminar la rejilla completa.

Se corrió varias veces seguidas hasta completar las 2,320 mediciones, guardadas en `datos/resultados.csv`.

5. Analizar los resultados y hacer las gráficas

Con los datos crudos, un script en Python (`analizar.py`) calcula el promedio de las 10 repeticiones de cada configuración, calcula el *speedup* de cada una contra la serial del mismo tamaño, y genera 5 gráficas que muestran: speedup vs. número de hilos, speedup vs. tamaño del problema, speedup de P2 vs. k, tiempo de ejecución vs. tamaño, y una comparación directa P1 vs. P2.

Resultado principal: ambas versiones paralelas superan el 1.5x de *speedup* que exigía el proyecto (promedio 1.62x en P1, 1.48x en P2 —con máximos de hasta 2.56x—), así que no hay penalización por ese lado. La NOTA 1 solo exige que AL MENOS UNA versión supere 1.5x, y aquí ambas lo hacen cómodamente en la mayoría de las configuraciones.

6. Escribir el reporte formal

Con los resultados ya en mano, se escribió el documento con todas las secciones que pide el PDF: descripción del código y de las estrategias de paralelización, definición del experimento, descripción del equipo de hardware/software usado, interpretación de los resultados, sección de uso responsable de IA generativa, y conclusiones.

Dónde quedó todo

```
dbscan_proyecto/
  src/            <- el código C++ (serial, P1, P2) y el generador de puntos
  Makefile        <- "make" compila los 3 programas
  experimentos/   <- los scripts que corrieron el experimento y el análisis
  datos/          <- los CSV: puntos de entrada, resultados crudos, promedios, speedups
  graficas/       <- las 5 gráficas en PNG
  pdfs/           <- este documento, el guión de presentación y la explicación técnica
```

El **reporte formal completo** (el que combina todas las secciones que pide el PDF, con las gráficas incrustadas) quedó como un documento aparte, en un link que les pasó José —no vive en esta carpeta, pero se puede exportar a PDF o Word desde ahí cuando esté listo para entregar—.

Lo que falta que revisen ustedes dos

1. ~~Confirmar el formato de `DBSCAN_noise.ipynb`~~ — **YA RESUELTO el 17 de octubre**: se verificó contra el código real de la libreta y se corrigió `generar_puntos.py` para igualarlo exactamente.
2. **Entender a fondo el código**, sobre todo la parte del *merge* en `dbscan_p2.cpp` (es la más sutil) —el documento "Explicación técnica" de esta misma carpeta va función por función y parámetro por parámetro, úsenlo para repasar—.
3. **Completar el nombre completo de Natalia** en la portada del reporte formal.
4. Repasar el **guión de presentación** (otro PDF en esta misma carpeta) y decidir quién dice qué parte.